Data Science Theory

An intuitive guide from probability to production

Andres Gonzalez Ortega • April 26, 2026

Contents

How to Read This Document 3

Part I: Mathematical Foundations 3

Probability and Statistical Estimation 3

Optimization and Loss Functions 4

A Note on the Glossary 5

Part II: Generalization, Validation, and Feature Engineering 6

Bias, Variance, and the Generalization Gap 6

Regularization and the Art of Not Overfitting 6

Cross-Validation Strategies 7

Data Leakage Patterns 8

Feature Engineering 9

Part III: Inference, Causality, and Uncertainty 10

Randomized Experiments and A/B Testing 10

Observational Causal Inference 11

Calibration and Conformal Prediction 11

Part IV: Interpretability, Fairness, and Governance 12

Model Interpretability with SHAP and Friends 12

Fairness and Responsible AI 13

Part V: Modern Technique Families 14

Deep Learning Fundamentals 14

LLM Foundations for Data Scientists 15

Computer Vision Foundations 16

Time Series Foundations 17

Recommender System Foundations 18

Part VI: Production and Infrastructure 19

MLOps Architecture Patterns 19

Cloud Deployment for ML Services 20

SQL for Data Scientists 21

Cheat Sheet 22

How to Read This Document

This document uses three colored callout boxes. Each color carries a fixed meaning, so the rest of the document leaves the boxes untitled — the color alone tells you which kind of content you are reading.

The crisp definition of the idea. Answers *what* the concept is.

A familiar situation that maps onto the concept. Answers what the concept *feels like*.

The picture to keep in mind when you recall the idea later. Answers how to *remember* it.

Part I: Mathematical Foundations

Probability and Statistical Estimation

Before fitting any model, it is worth asking: what does it mean for an estimate to be good? Maximum likelihood estimation gives one precise answer, and the answer has a beautiful geometric interpretation. How confident should we be in any particular estimate, and where does that confidence actually come from?

Maximum likelihood estimation chooses the parameter value that makes the observed data most probable. Its key theoretical payoff is asymptotic normality: as the sample grows, the MLE concentrates around the true value at a rate governed by the Fisher information, and that concentration justifies standard errors and Wald confidence intervals across virtually all of applied statistics.

Think of the likelihood function as a landscape, and the MLE as the summit. Fisher information measures how sharply peaked that summit is. A sharp peak means the data pins down the parameter tightly; a broad, flat hill means many parameter values fit almost equally well, so any single estimate is uncertain. The

variance of the MLE is literally the reciprocal of that curvature.

$$\sqrt{n}\left(\hat{\theta}_{\text{MLE}} - \theta_0\right) \xrightarrow{d} \mathcal{N}\left(0,\ I(\theta_0)^{-1}\right)$$

Here $I(\theta_0)$ is the Fisher information matrix evaluated at the true parameter and n is the sample size.

The bridge from MLE to actuarial credibility runs through the normal-normal conjugate model. The Buhlmann credibility formula $Z\bar{X}_i + (1-Z)\mu$ is precisely the Bayesian posterior mean; MLE estimates the hyperparameters (μ, k) from the portfolio data. Asymptotic normality is the frequentist road to the same destination.

Optimization and Loss Functions

Every machine learning model is, at its core, a minimization problem. The choice of loss function decides what the model is actually optimized to do; the choice of optimizer decides how efficiently it gets there. Have you ever wondered why two models with identical architectures can behave so differently in practice?

Every ML model is defined by $\hat{\theta} = \arg\min_\theta L(\theta; \text{data})$. The loss function and optimizer are not implementation details; they determine what the model is solving and whether it will find a good solution. Changing the loss can be more impactful than changing the architecture.

Picture SGD momentum descending a narrow, steep-sided ravine. Without momentum, the gradient oscillates back and forth across the high-curvature walls and makes slow progress along the valley floor. With momentum, the running average of gradients damps those side-to-side oscillations and amplifies the consistent downhill direction, reaching the bottom far more quickly.

The AdamW update combines adaptive learning rates with decoupled weight decay:

$$\theta_{t+1} = \theta_t - \eta\left(\frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} + \lambda\,\theta_t\right)$$

where $\hat{m}_t$ and $\hat{v}_t$ are bias-corrected first and second gradient moments and $\lambda\,\theta_t$ is the

decoupled weight-decay term that penalizes large weights independently of the gradient magnitude.

The condition number κ of the loss surface is its aspect ratio: a wide, shallow bowl in one direction and a steep wall in another. Adaptive optimizers implicitly rescale each parameter's learning rate by the inverse of its recent gradient magnitude, transforming that ill-conditioned bowl into something closer to a sphere, where every direction is equally easy to traverse.

A Note on the Glossary

Much confusion in data science comes from the same word meaning different things in different communities. A "model" in statistics is a probability distribution; in machine learning it is a function from inputs to predictions; in business it is any analytical tool used to make a decision. Which definition is in play matters more than we usually admit.

Precision in language is precision in thinking. Before presenting results to a new audience, anchor every key term: what is the outcome variable, what counts as a prediction, and what criterion defines success. A number without a unit is meaningless; a metric without a definition is worse, because it creates false confidence.

Consider the word "significant." In statistics it means the p-value fell below a pre-specified threshold; in business it means the effect is large enough to act on; in casual speech it means noteworthy. The same word in a board presentation and in a methods section is doing completely different work.

Keep a one-page glossary for every project and share it at the first stakeholder meeting. The questions it provokes are the most valuable part of the exercise: they surface implicit disagreements about what is actually being measured before anyone has spent a week modeling the wrong thing.

Part II: Generalization, Validation, and Feature Engineering

Bias, Variance, and the Generalization Gap

A model that memorizes the training data perfectly is not necessarily a good model. The central question is always: how does it perform on data it has never seen? The bias-variance decomposition makes that question mathematically precise.

For squared-error loss, expected prediction error decomposes into three parts: bias squared (systematic error that persists across different training sets), variance (how much the model fluctuates as training data changes), and irreducible noise from the world itself. Reducing one generally increases another; there is no free lunch.

Imagine three groups of archers at a target. The high-bias group clusters all arrows together, but consistently far from the bullseye. The high-variance group spreads arrows widely in every direction. The well-calibrated group clusters near the center. Good modeling is the third group: consistent and accurate.

$$E\Big[(y - \hat{f}(x))^2\Big] = \Big(E[\hat{f}(x)] - f(x)\Big)^2 + \mathrm{Var}[\hat{f}(x)] + \sigma^2$$

The expectation is taken over the randomness in both the training data and the label noise σ^2.

The expectation in the decomposition is over hypothetical re-draws of entire training sets, not over individual noisy labels. Bias and variance describe properties of the learning algorithm across many possible datasets, not the behavior of one fitted model on one dataset. That distinction is why you cannot estimate bias from a single run.

Regularization and the Art of Not Overfitting

Regularization often feels like an arbitrary penalty added to the loss. It is not arbitrary at all: it is a prior belief about the size of coefficients, expressed in the language of

optimization. Where do your priors come from, and what do they imply about the penalties you add?

L2 regularization is MAP estimation under a Gaussian prior on the coefficients; L1 is MAP under a Laplace prior. The penalty strength $\lambda = \sigma^2/\tau^2$ is the ratio of noise variance to prior variance, exactly analogous to the complement $1 - Z$ in Buhlmann credibility. Regularization is Bayesian shrinkage written in optimization notation.

Think of regularization as a complexity budget. L2 charges quadratically: spreading weight across many features is affordable because each small weight costs little. L1 charges linearly: a single large weight is as expensive as many small ones, so the budget pressure pushes toward sparse solutions where most weights land at exactly zero.

The log-posterior under a Gaussian prior recovers Ridge regression:

$$\log \pi(\beta \mid y) \propto -\frac{1}{2\sigma^2}\|y - X\beta\|_2^2 - \frac{1}{2\tau^2}\|\beta\|_2^2$$

Setting $\lambda = \sigma^2/\tau^2$ gives the standard Ridge objective.

Ridge regression is multivariate Buhlmann credibility: each coefficient is shrunk toward zero by the ratio of noise variance to prior variance, just as an individual claim mean is blended toward the portfolio mean in proportion to how much data supports it. The math is identical; only the vocabulary differs.

Cross-Validation Strategies

Cross-validation is easy to run and easy to misinterpret. The metric it returns is only as trustworthy as the splitting scheme used to produce it. Have you checked whether your splits respect the actual structure of your data?

Cross-validation averages held-out performance estimates to get a low-variance proxy for generalization error. The critical requirement is that the splitting scheme must reflect how the model will be used in production. A split that does not match the deployment structure produces an estimate that is confidently

wrong, which is worse than no estimate at all.

Each fold is a dress rehearsal under one specific scenario. The mean score across folds is the expected performance; the spread across folds tells you how fragile the model is to which slice of data happens to be held out. A low mean with high spread signals that performance is driven by a few easy subsets, not by general learning.

A key theoretical result due to Bengio and Grandvalet (2004) is that no general unbiased estimator of the variance of k-fold CV error exists. The fold standard deviation should be treated as a stability indicator, not as a classical confidence interval.

For time series, the arrow of time is inviolable. Purged and embargoed cross-validation adds a gap at fold boundaries to prevent features or labels computed near the boundary of one fold from bleeding information into the adjacent fold. Skipping this step produces optimistically biased estimates that collapse the moment the model is deployed.

Data Leakage Patterns

Data leakage is the most reliably damaging mistake in applied machine learning, and it is easy to commit without noticing. The diagnostic question is deceptively simple: at the moment of prediction in production, exactly what information is available?

Anything used during training that would not be available at prediction time is leakage. Leakage causes offline metrics to overstate real-world performance, sometimes dramatically. The model has learned to exploit an information shortcut that disappears the moment it is deployed.

Imagine an exam where the answer key was accidentally included in the exam packet. The student scores perfectly, but learned nothing about the subject. Offline AUC looks excellent because the model memorized the leak, not the underlying pattern. The course correction feels brutal precisely because the misleading signal was so convincing.

The canonical fix is a scikit-learn `Pipeline` that fits all transformers only on training folds, never on the validation fold. Point-in-time feature stores enforce temporal cutoffs

at the row level, ensuring that no feature for a given row uses information from after the prediction timestamp.

A too-good-to-be-true offline metric, such as AUC 0.99 on a genuinely hard real-world problem, is the primary warning sign. A single feature that explains almost everything is the secondary one. Either observation should trigger an immediate audit of the data pipeline, not a celebration.

Feature Engineering

Raw columns rarely tell the whole story. Feature engineering is the systematic process of asking: what does an expert know that the data does not yet express? The answer to that question is often the difference between a mediocre model and a great one.

Features encode domain knowledge in a form the model can exploit. The question is always whether the transformation reveals structure that the raw data obscures. Target encoding and actuarial credibility are the same estimator: both shrink a group mean toward a global mean by an amount proportional to how much data the group contains.

The smoothed target encoding for category c is $\hat{y}_c = Z_c \bar{y}_c + (1 - Z_c)\bar{y}$, where $Z_c = n_c/(n_c + k)$. This is Buhlmann's formula with $k = \sigma_\varepsilon^2/\sigma_\mu^2$: the credibility weight grows as the group provides more observations and shrinks when the group is noisy relative to the portfolio.

Weight of Evidence and Information Value are classical credit-scoring features that generalize to any binary classification problem:

$$\mathrm{WoE}_c = \log \frac{P_c}{N_c}, \qquad \mathrm{IV} = \sum_c (P_c - N_c) \cdot \mathrm{WoE}_c$$

where P_c and N_c are the shares of positives and negatives falling in category c.

WoE converts each category to a log-odds score: positive values signal enrichment for positive outcomes, negative values signal depletion. IV sums these per-category log-odds signals, weighted by how common each category is, into a single number that summarizes how much discriminatory power a feature carries across all its

levels.

Part III: Inference, Causality, and Uncertainty

Randomized Experiments and A/B Testing

Correlation shows up everywhere in product data. Causation is rarer, and only one method establishes it by construction. What is special about randomization, and why does it justify causal claims that no amount of observational data can?

Randomization breaks the dependence between treatment assignment and every possible confounder, observed or unobserved. As a result, the observed difference in outcomes between treatment and control is an unbiased estimate of the average treatment effect. No other study design achieves this without additional assumptions.

An A/B test is a controlled scientific experiment embedded in a live product. The discipline of a good experiment is mostly pre-experiment work: pre-register the hypothesis, metric, minimum detectable effect, required duration, and decision rule before the test starts. Any analysis conducted after peeking at results is exploratory, not confirmatory.

The required sample size per arm for a two-sample test is:

$$n_{\text{per arm}} = \frac{2\,(z_{1-\alpha/2} + z_{1-\beta})^2\,\sigma^2}{\delta^2}$$

where δ is the minimum detectable effect, σ^2 is the outcome variance, and $z_{1-\alpha/2}$, $z_{1-\beta}$ are the normal quantiles for the chosen Type I and Type II error rates.

CUPED variance reduction is essentially free. Residualizing the outcome on a pre-experiment covariate reduces variance by a factor of $1 - \rho^2$. At $\rho = 0.7$, a 20,000-user experiment shrinks to roughly 10,200 users for the same power, cutting the required runtime nearly in half with no additional cost.

Observational Causal Inference

We cannot always run experiments. Treatment may be unethical to withhold, impossible to control, or already baked into historical data. Observational causal inference asks: under what assumptions can we recover causal effects from data that was not randomized?

Potential outcomes $Y(1)$ and $Y(0)$ are never simultaneously observed for the same unit. Causal identification requires assumptions: parallel trends for difference-in-differences, ignorability for propensity score methods, or an exclusion restriction for instrumental variables. The assumptions are the argument; the method is just the arithmetic that follows from them.

A directed acyclic graph is the causal map you draw before touching data. Nodes are variables, arrows are direct causal influences. Backdoor paths from treatment to outcome through confounders are what create spurious associations. The method you choose should be the one that blocks those paths given what you can observe and control.

The difference-in-differences estimator is:

$$\hat{\tau}_{\text{DiD}} = \left(\bar{Y}_{T,\text{post}} - \bar{Y}_{T,\text{pre}}\right) - \left(\bar{Y}_{C,\text{post}} - \bar{Y}_{C,\text{pre}}\right)$$

Identification requires that, absent treatment, the treatment group would have followed the same trend as the control group (parallel trends).

CausalImpact uses a Bayesian structural time series model to construct a synthetic counterfactual from the pre-period and a set of donor series. This is DiD for a single treated time series: the pre-period model is the counterfactual; the post-period gap between actuals and the synthetic control is the estimated treatment effect.

Calibration and Conformal Prediction

A model that ranks cases perfectly can still be wrong about its probabilities. Calibration and discrimination are independent properties, and real decisions often require both.

Would you trust a model that says "90% confident" when it is right only 60% of the time?

A model is calibrated if, among all examples assigned probability p, a fraction p are actually positive. Calibration is a global property of the probability outputs, not of the ranking. A model can have excellent AUC and terrible calibration, or the reverse. For risk pricing and reserve setting, calibration is not optional.

A weather forecaster who says "70% chance of rain" should be right on 70% of the days they make that call. The reliability diagram plots actual frequency against predicted probability across confidence bins; a perfectly calibrated model falls on the diagonal. ECE summarizes the average gap into one number:

$$\text{ECE} = \sum_{m} \frac{|B_m|}{n} \left|\text{acc}(B_m) - \text{conf}(B_m)\right|$$

where B_m is a confidence bin, $\text{acc}(B_m)$ is the fraction positive in that bin, and $\text{conf}(B_m)$ is the mean predicted probability.

Conformal prediction provides the following finite-sample guarantee under exchangeability alone, with no model assumptions: $P(Y_{n+1} \in C(X_{n+1})) \geq 1 - \alpha$. No distributional assumption is needed; only exchangeability. That makes conformal prediction sets the most defensible form of uncertainty quantification for a regulatory or audit context.

Part IV: Interpretability, Fairness, and Governance

Model Interpretability with SHAP and Friends

Knowing a model is accurate is not the same as knowing why it is accurate. For decisions that affect people, the explanation matters as much as the prediction. SHAP values offer one mathematically principled way to distribute credit among features.

SHAP values are the unique attribution satisfying four Shapley axioms: efficiency (contributions sum to the prediction minus the baseline), symmetry (equal contributors get equal credit), null player (features that change nothing get zero), and additivity (SHAP values of combined models sum correctly). Efficiency is the one most practitioners care about in production: every gap between prediction and baseline is fully explained.

Imagine a group of colleagues who collaborated on a project with a measurable outcome. Each colleague's fair share is their average marginal contribution: how much better the outcome gets when they are added to every possible subset of the team, averaged over all orderings. SHAP applies exactly that fairness principle to features in a model.

$$\phi_j = \sum_{S \subseteq F \setminus \{j\}} \frac{|S|!\,(|F| - |S| - 1)!}{|F|!} \Big[v(S \cup \{j\}) - v(S) \Big]$$

where $v(S)$ is the expected model prediction when only features in S are known and F is the full feature set.

Conditional SHAP and interventional SHAP give different answers when features are correlated. Conditional SHAP respects the data distribution; interventional SHAP treats features as independent. For regulatory adverse-action letters, document which value function you used and why, because the choice changes which features receive credit.

Fairness and Responsible AI

There is no single, universally agreed definition of fairness. That is not a philosophical footnote; it is a practical constraint that forces a decision before any model is deployed. Which definition you target, and why, is a normative choice that belongs to the humans involved, not to the model.

Demographic parity, equal opportunity, equalized odds, and predictive parity are mutually incompatible whenever base rates differ across groups. Satisfying one definition mathematically precludes satisfying the others. There is no technical fix; the tension is fundamental, and the resolution requires a policy decision about

which errors are more acceptable.

Intersectional analysis reveals gaps that single-attribute analysis hides. A model can satisfy the 80% rule on gender and separately on ethnicity, while producing substantially worse outcomes for a specific intersection such as indigenous women, than any one-dimensional audit would detect. Fairness audits that ignore intersections are incomplete by construction.

Differential privacy provides a formal bound on the information a model reveals about any individual record:

$$P(M(D) \in S) \leq e^{\epsilon}\, P(M(D') \in S) + \delta$$

where D and D' differ in exactly one record. Smaller ϵ means stronger privacy guarantees; δ is the probability of a catastrophic failure of the bound.

Fairness is not a setting to enable; it is a conversation to conduct. Declare which definition you target and justify it. Measure the gap per group before and after any intervention. Document the accuracy-fairness tradeoff explicitly. The documentation is evidence that the decision was made deliberately rather than by default.

Part V: Modern Technique Families

Deep Learning Fundamentals

Neural networks looked like they would require enormous amounts of hand-crafted gradient code until automatic differentiation changed everything. What made deep learning suddenly tractable was not just more data or faster chips; it was backpropagation built into software that could handle arbitrary computation graphs.

Backpropagation is the chain rule of calculus applied systematically to a computational graph. Modern autograd frameworks build the graph during the forward pass and traverse it backward automatically. The user writes the forward computation; the framework computes all gradients without any user-written derivative code.

A ResNet can be understood as an exponential ensemble of shallow paths. Each residual block provides an identity shortcut: the block's output can be skipped entirely or added to the learned transformation, giving 2^L implicit subnetworks across L blocks. Gradients always have a direct channel back through the identity paths, which is why very deep ResNets remain trainable.

LoRA (Low-Rank Adaptation) fine-tunes a large model by adding a low-rank update to each weight matrix:

$$W' = W + \Delta W, \quad \Delta W = BA, \quad A \in \mathbb{R}^{r \times n},\ B \in \mathbb{R}^{m \times r},\ r \ll \min(m, n)$$

Only $r(m+n)$ parameters are trained instead of mn, reducing memory and compute by orders of magnitude for large weight matrices.

Training a deep network is like teaching a chain of translators: each layer translates the representation from the previous layer into something more useful for the next. Residual connections add a direct phone line so that error signals can reach every layer without being distorted by passing through every intermediate translation step.

LLM Foundations for Data Scientists

Large language models are not magic; they are transformers trained on next-token prediction at a scale that produces emergent capabilities. Understanding their training pipeline helps you decide when to use them, how to adapt them, and where they will fail.

LLMs are trained in stages: pretraining on next-token prediction over vast corpora, supervised fine-tuning on instruction-response pairs, alignment via RLHF or DPO to match human preferences, and domain adaptation via low-rank methods such as LoRA. Each stage adds a layer of behavior on top of the statistical foundation built by pretraining.

Speculative decoding is free speed. A small draft model proposes k tokens cheaply; the large target model verifies all k in a single forward pass, which costs roughly the same as generating one token. Accepted tokens commit; the first rejection

rolls back. The result is a 2x to 3x latency reduction with no change in output quality.

Scaled dot-product attention is the core computational primitive:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$

The $\sqrt{d_k}$ scaling prevents dot products from growing so large that they push the softmax into a saturated regime where gradients vanish.

Attention is a soft database lookup. The query says "what am I looking for"; the keys say "what each stored entry is about"; the values say "what each entry actually contains." The softmax computes a weighted blend: entries whose keys match the query closely contribute most to the output.

Computer Vision Foundations

Image pixels are not meaningful in isolation. What matters is the spatial relationship between pixels: an edge, a texture, a shape. Convolutional layers exploit this structure explicitly, and the resulting visual hierarchy turns out to transfer remarkably well across domains.

Convolutional layers are local, weight-shared, and parameter-efficient. Each filter learns to detect a particular pattern anywhere in the image. As the network deepens, filters at higher layers respond to increasingly abstract patterns: edges, then textures, then parts, then objects. This hierarchy is the source of both the efficiency and the transferability of convolutional networks.

CLIP's zero-shot classification illustrates how vision and language can be aligned without task-specific training. Encode each candidate label as "a photo of a [class]" and encode the input image. Predicted class is the label with highest cosine similarity. No labeled examples of the new class are required; the shared embedding space does the work.

Diffusion models learn to generate images by reversing a forward noising process. One step of the forward process is:

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(x_t;\ \sqrt{1-\beta_t}\, x_{t-1},\ \beta_t I\Big)$$

After T steps, the image becomes approximately isotropic Gaussian noise; the network learns to denoise backward from T to 0.

Transfer learning works because the visual hierarchy from edges to objects is domain-general. The features learned on ImageNet are not about ImageNet; they are about the geometry of natural images. Only the final mapping from abstract visual concepts to your specific class labels needs to be retrained.

Time Series Foundations

Time series data has one feature that distinguishes it from every other data type: the order of rows encodes information. Shuffle a tabular dataset and you still have the same dataset; shuffle a time series and you have destroyed it. That ordering constraint permeates modeling, evaluation, and feature engineering.

Time series observations are sequentially ordered; swapping rows changes the data. This violates the IID assumption underlying most off-the-shelf ML methods and requires specialized evaluation (forward-chaining CV), modeling (ARIMA, GARCH, neural sequence models), and feature engineering (lag features, rolling statistics, Fourier terms).

GARCH is a weather model for volatility. Calm days tend to be followed by calm days; stormy days tend to be followed by stormy days. The model tracks the running mood of the market and adjusts the expected variance for the next period based on recent shocks and recent conditional variances.

The GARCH(p, q) conditional variance equation:

$$\sigma_t^2 = \alpha_0 + \sum_{i=1}^{q} \alpha_i\, \varepsilon_{t-i}^2 + \sum_{j=1}^{p} \beta_j\, \sigma_{t-j}^2$$

Stationarity of the variance process requires $\sum_i \alpha_i + \sum_j \beta_j < 1$.

Hierarchical forecasting is the problem: a multi-level structure such as nation, region, store, SKU must produce forecasts that add up consistently. Forecast reconciliation is the family of solutions. The MinT method chooses reconciliation weights that minimize the total forecast-error variance across all levels of the hierarchy.

Recommender System Foundations

Recommender systems face a data problem that most classification tasks do not: the absence of an interaction does not mean the user dislikes the item. It may simply mean the user never saw it. How you handle that asymmetry between observed and unobserved interactions shapes everything downstream.

Implicit feedback such as clicks and purchases is abundant but ambiguous. Unobserved items are not negatives; they are missing. Pairwise losses that require only relative ordering between observed and unobserved items are better suited to this data structure than pointwise MSE, which treats every unobserved item as a zero-score target.

A two-stage recommender system resembles airport security. Retrieval, typically approximate nearest neighbor search in an embedding space, is the fast first pass that narrows millions of candidates to a shortlist of hundreds. Ranking is the thorough second pass that scores each candidate carefully using a richer feature set.

Bayesian Personalized Ranking optimizes for the probability that observed items rank above unobserved items:

$$\mathcal{L}_{\text{BPR}} = \sum_{(u,i,j) \in D_S} \log \sigma(\hat{x}_{ui} - \hat{x}_{uj}) - \lambda \|\Theta\|^2$$

where i is an observed interaction, j is an unobserved item, and σ is the sigmoid function.

Off-policy evaluation via inverse propensity scoring is the same reweighting technique as in observational causal inference. Logged interactions were generated under a historical recommendation policy; IPS reweights them to estimate what a

new policy would have produced, allowing you to evaluate a new ranker without running a live experiment.

Part VI: Production and Infrastructure

MLOps Architecture Patterns

A model that works in a notebook and a model that works in production are different objects. The gap between them is the subject of MLOps: not just deployment, but monitoring, retraining triggers, data versioning, and feature consistency. What breaks first when a model moves from a researcher's laptop to a production API?

Training-serving skew is the most common silent production failure. Features computed differently at training time versus inference time cause the model to receive inputs outside its training distribution without any visible error. A feature store with point-in-time correctness is the primary defense: it enforces that training and serving retrieve features using identical logic.

Shadow deployment is the safest way to evaluate a candidate model. Every production request is duplicated; the candidate model receives a copy and produces a prediction that is logged but never acted on. Users see no change; you accumulate full-fidelity production signal to compare the candidate against the current model before any traffic is shifted.

Population Stability Index measures distribution shift between training and production:

$$\text{PSI} = \sum_i (p_i - q_i) \log \frac{p_i}{q_i}$$

where p_i is the training-time proportion and q_i is the current production proportion in bin i. PSI above 0.2 is the conventional threshold for investigation.

MLOps maturity is a step function, not a slope. Maturity 0 is deploying a notebook manually. Maturity 1 is an automated training and deployment pipeline

triggered by new data. Maturity 2 is CI/CD for the model itself: code changes trigger tests, performance regressions trigger rollbacks. Each level is a qualitative improvement in production reliability.

Cloud Deployment for ML Services

A trained model sitting in a file has zero business value. Converting it into a service that responds to requests, scales with demand, and recovers from failures is the deployment problem. What does a minimal but production-grade deployment look like, and what does each component actually protect against?

Cloud deployment converts a trained artifact into a service with autoscaling, monitoring, and rollback. Without these capabilities, the model cannot serve users reliably, cannot adapt to load changes, and cannot recover from errors without manual intervention. Deployment is not the end of the ML workflow; it is where the workflow begins to matter.

A multi-service request taking 2.5 seconds may spend 200 ms inside the model and 2.3 seconds waiting for a misconfigured database view. Without distributed tracing you blame the model; with OpenTelemetry traces you see the actual bottleneck in the waterfall and fix it in minutes rather than days of speculation.

A minimal production deployment can be thought of as five concentric reliability rings: the Docker image (reproducibility), Cloud Run (autoscaling and managed infrastructure), GitHub Actions CI/CD (automated testing and deployment), a monitoring dashboard (observability), and budget alerts (cost control). Each ring catches a different class of failure that the inner rings cannot prevent.

Cold-start latency is the main user-experience risk in serverless deployment. Mitigations layer: lazy model loading defers expensive work until the first request, startup probes prevent traffic from reaching an unready instance, provisioned concurrency keeps warm instances ready, and model-server warmup caches predictions for common inputs. Together, these keep p99 latency within SLA even when instances spin up from zero.

SQL for Data Scientists

SQL is four decades old and remains the most widely used language for data manipulation in production. Its staying power comes from a design choice that is easy to underestimate: SQL is declarative. You describe the result; the query planner decides how to compute it. Understanding the planner is what separates a query that runs in seconds from one that runs for hours.

SQL is declarative: you specify the shape of the result, not the steps to compute it. The query planner translates your intent into a physical execution plan. Your job is to help the planner by providing accurate schema information, appropriate indexes, and query shapes that expose the structure the planner can exploit, such as filter pushdown and partition pruning.

The ROWS versus RANGE distinction in window functions is invisible until there are tied values. Consider three orders placed on the same date. ROWS treats each row as a separate point in the frame, counting them one by one. RANGE treats all rows with the same ORDER BY value as a peer group and includes all of them simultaneously. The results diverge exactly when ties exist and exactly where correct lifetime-value calculation matters most.

The standard running-total window function uses a ROWS frame: `SUM(revenue) OVER (PARTITION BY customer_id ORDER BY order_date ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)`. This computes the running lifetime value per customer, adding each order one row at a time and never including future rows in the cumulative total.

`EXPLAIN ANALYZE` shows the plan the query planner actually chose and the real row counts at each step. A sequential scan on a large table indicates a missing index. A row-count estimate that is orders of magnitude from the actual count indicates stale statistics; running `ANALYZE` on the table lets the planner rebuild its estimates and often halves execution time without changing the query.

Cheat Sheet

Concept	One-Line Model	Picture
Probability & MLE	$\sqrt{n}(\hat{\theta} - \theta_0) \to \mathcal{N}(0, I^{-1})$	Sharpness of likelihood peak
Optimization & Loss	AdamW: adaptive rates with decoupled weight decay	Condition number = bowl aspect ratio
Glossary	Precision in language is precision in thinking	Shared glossary prevents week-long wrong models
Bias-Variance	$E[(y - \hat{f})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$	Three archers: scattered, biased, centered
Regularization	L2 = Gaussian MAP; L1 = Laplace MAP; $\lambda = \sigma^2/\tau^2$	Complexity budget charged quadratically or linearly
Cross-Validation	Mean score = performance; spread = fragility	Dress rehearsal under each scenario
Data Leakage	Available at prediction time? If not, it is leakage.	Answer key in the exam packet
Feature Engineering	WoE, target encoding = Buhlmann credibility	Expert knowledge encoded as a number
A/B Testing	$n = 2(z_{\alpha/2} + z_\beta)^2\sigma^2/\delta^2$	Pre-registered scientific experiment in a product
Causal Inference	DiD = double difference; DAG = causal map	Block backdoor paths before touching data
Calibration	ECE = mean gap between confidence and accuracy	Weather forecaster: 70% call should hit 70% of the time
SHAP	ϕ_j = average marginal contribution over all subsets	Team credit split fairly by marginal contribution
Fairness	No single definition; declare which one and measure gaps	Intersections hide what single attributes conceal
Deep Learning	Backprop = chain rule on computation graph	Chain of translators with direct phone lines (residuals)
LLM Foundations	Attention = soft database lookup; LoRA = low-rank update	Pretraining + SFT + alignment + domain adaptation
Computer Vision	Conv layers: local, shared, hierarchical	Visual hierarchy transfers; only the head needs retraining
Time Series	GARCH(p, q): $\sigma_t^2 = \alpha_0 + \sum \alpha_i \epsilon_{t-i}^2 + \sum \beta_j \sigma_{t-j}^2$	Volatility clusters; order of rows encodes information
Recommender Systems	BPR: observed items should rank above unobserved	Two-stage: retrieval (fast) then ranking (careful)
MLOps	PSI > 0.2 triggers drift investigation	Maturity 0: notebook; 1: pipeline; 2: CI/CD for model